Firjan SENAI

Técnico

JavaScript

CONCEITOS • VARIÁVEIS • ESTRUTURAS CONDICIONAIS • LAÇOS DE REPETIÇÃO • ALGORITMOS



2025



Sumário

1. **Introdução à Lógica de Programação**
   O que é lógica de programação e por que aprender
2. **Tipos de Dados e Variáveis**
   Números, textos, booleanos e declaração de variáveis
3. **Operadores e Expressões**
   Aritméticos, relacionais e lógicos
4. **Estruturas Condicionais**
   Decisões com if, else e switch
5. **Laços de Repetição**
   Repetição com for, while e do...while
6. **Algoritmos**
   Representação e construção de soluções passo a passo
7. **Funções e Modularização**
   Organização do código em blocos reutilizáveis



04

Funções

Desenvolvimento de Sistemas - Javascripts
Instrutor: Vitor Hugo Almeida de Sousa

Firjan SENAI

JavaScript - Funções

Funções na programação são blocos de código reutilizáveis que executam uma tarefa específica. Elas permitem organizar melhor o programa, evitando repetições e facilitando a manutenção. Uma função pode receber valores de entrada (parâmetros), realizar operações com esses dados e, opcionalmente, retornar um resultado. Essa estrutura torna o código mais claro, modular e eficiente.

Segue um exemplo de código JavaScript utilizando função

- Sintaxe

```javascript
// Função que exibe uma saudação no console
function mostrarSaudacao() {
    var mensagem = "Olá! Seja bem-vindo ao mundo da programação!";
    console.log(mensagem);
}

// Chamando a função
mostrarSaudacao();
```

JavaScript

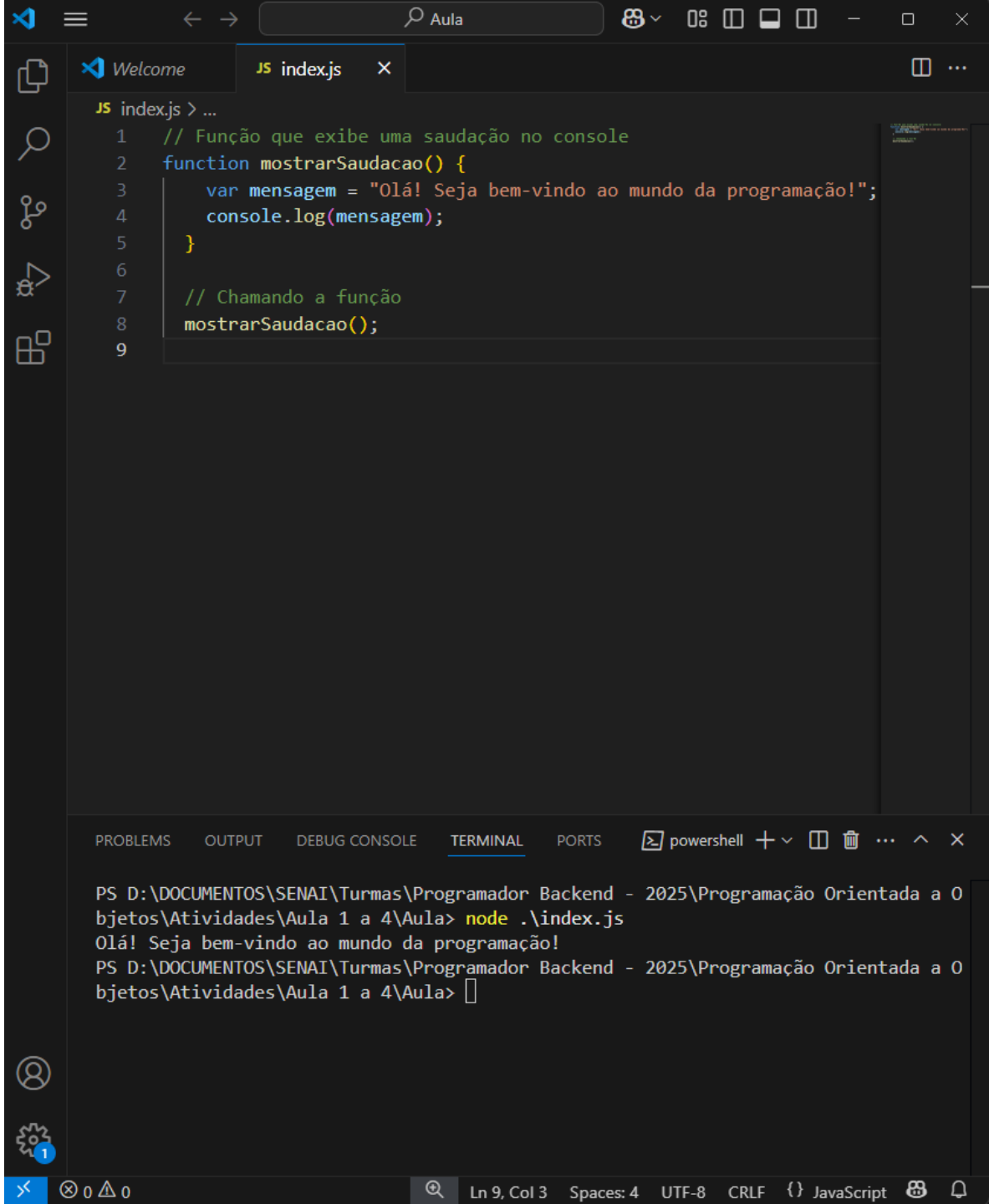
Funções

Objetivo: entender sintaxe das funções em JavaScript.

- Sintaxe

Exercícios

- Crie um arquivo chamado funcoes01.js dentro da pasta Aula 6;
- Crie um código na imagem ilustrada ao lado;
- Abra o Terminal e execute o código no VS Code e observe o resultado.



JavaScript

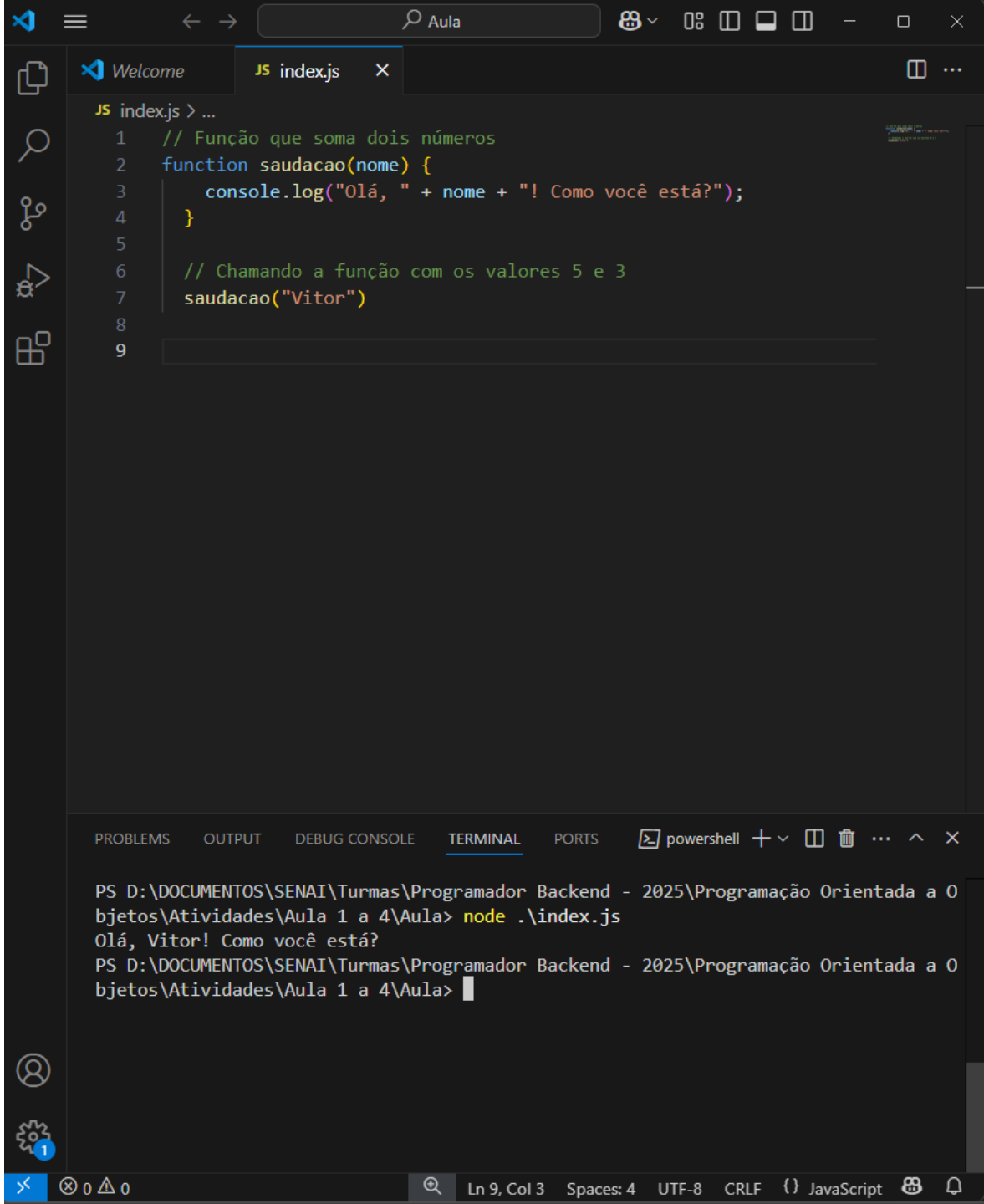
Funções

Objetivo: : entender parâmetros de funções em JavaScript.

- Sintaxe

Exercícios

- Crie um arquivo chamado funcoesComParametro01.js dentro da pasta Aula 3;
- Crie um código na imagem ilustrada ao lado;
- Abra o Terminal e execute o código no VS Code e observe o resultado.
- Chame a função mais 3 vezes e altere apenas os valores do parâmetro e veja o resultado.



JavaScript

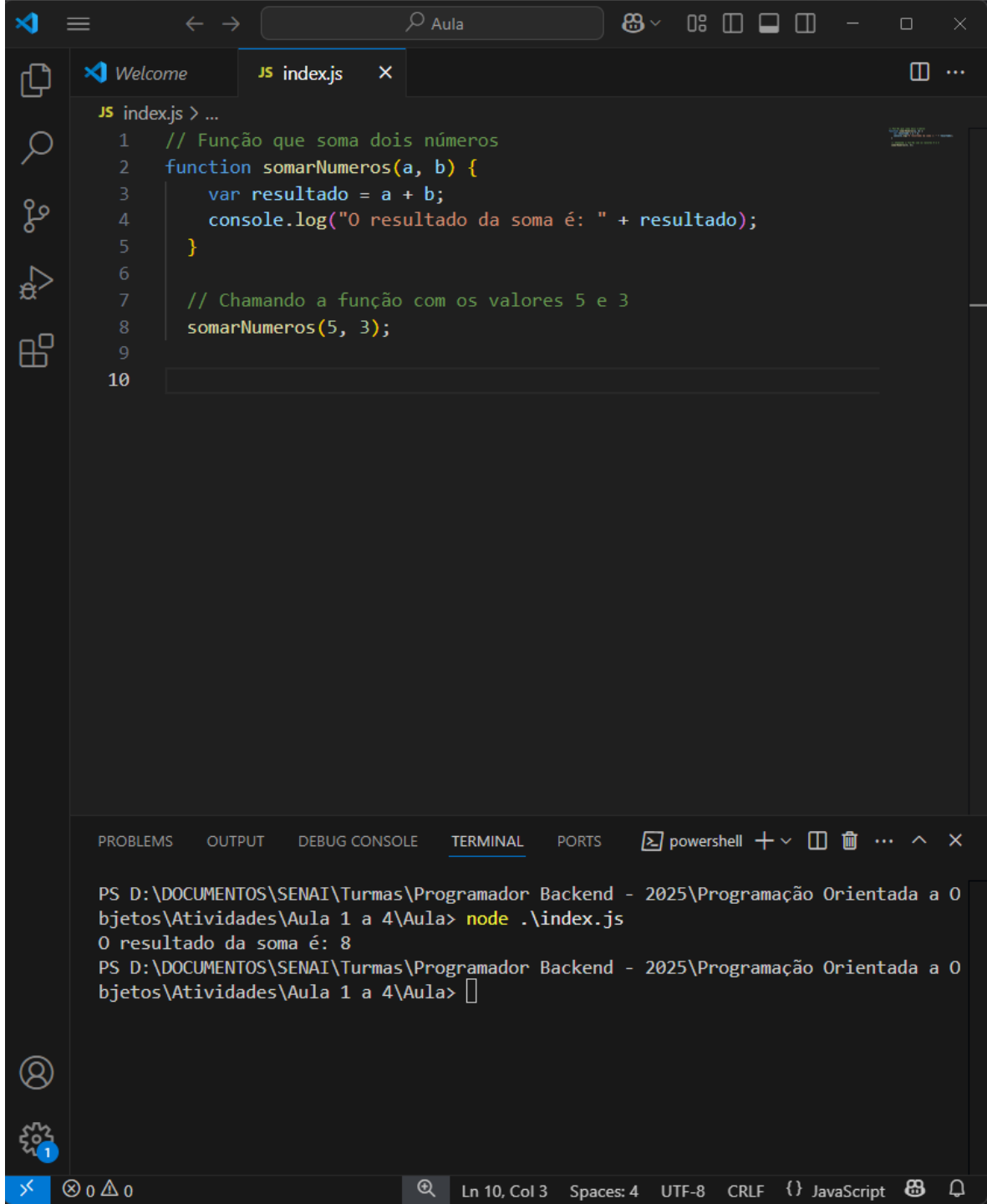
Funções

Objetivo: entender parâmetros de funções em JavaScript.

- Sintaxe

Exercícios

- Crie um arquivo chamado funcoesComParametro_02.js dentro da pasta Aula 3;
- Crie um código na imagem ilustrada ao lado;
- Abra o Terminal e execute o código no VS Code e observe o resultado.
- Chame a função somarNumeros mais 3 vezes passando como parâmetro números diferentes e veja qual é o resultado.



JavaScript

Função anônima

Uma função anônima é uma função escrita sem um nome próprio. Podemos guardar essa função em uma variável e usar o nome da variável para chamá-la.

A função foi escrita sem um nome depois de function. O nome somar pertence à variável que guarda a função. Ao escrever somar(3, 5), executamos a função guardada nela.

```
const somar = function (a, b) {
  return a + b;
};

const resultado = somar(3, 5);
console.log(resultado); // 8
```

JavaScript

Arrow function (=>)

Uma arrow function é uma forma de escrever uma função em JavaScript usando a seta =>. À esquerda da seta ficam os parâmetros. À direita fica o corpo da função, que diz o que ela faz.

Essa é a mesma soma do exemplo anterior. Mudou a forma de escrever a função. A chamada continua sendo somar(3, 5).

```
const somar = (a, b) => {
  return a + b;
};

console.log(somar(3, 5)); // 8
```

O termo lambda

Saiba Mais!



Você pode encontrar o termo lambda ao estudar funções curtas que podem ser guardadas em variáveis ou passadas para outras funções. Em JavaScript, a arrow function é uma forma comum de escrever esse tipo de função. O símbolo => faz parte da sintaxe da arrow function.

JavaScript

Retorno implícito

Quando o corpo da arrow function é uma única expressão sem chaves, o resultado dessa expressão é retornado automaticamente. Isso é chamado de retorno implícito.

```
const somar = (a, b) => a + b;

console.log(somar(3, 5)); // 8
```

Se usarmos chaves para abrir um bloco de instruções, precisamos escrever return para devolver o resultado.

```
const somar = (a, b) => {
  const total = a + b;
  return total;
};
```

Chaves sem return

Atenção



```
const somar = (a, b) => {
  a + b;
};

console.log(somar(3, 5)); // undefined
```

Nesse caso, a soma é calculada, mas o resultado não é devolvido para quem chamou a função.

JavaScript

Parâmetros antes da seta

Quando a função recebe um único parâmetro simples, os parênteses são opcionais. Com dois ou mais parâmetros, ou quando não há parâmetros, usamos parênteses.

```
const saudacao = () => "Olá!";
const dobrar = numero => numero * 2;
const somar = (a, b) => a + b;

console.log(saudacao()); // Olá!
console.log(dobrar(4));  // 8
console.log(somar(3, 5)); // 8
```

JavaScript

Callback com find()

Uma callback é uma função passada como argumento para outra função ou método. No find(), a callback informa qual condição cada elemento deve atender para ser encontrado.

O find() chama a função com um produto por vez. Para o mouse, produto.id === 2 retorna false. Para o teclado, retorna true. Nesse momento, o find() devolve o objeto do teclado e encerra a busca.

```
const produtos = [
  { id: 1, nome: "Mouse" },
  { id: 2, nome: "Teclado" },
  { id: 3, nome: "Monitor" }
];

const encontrado = produtos.find(produto => produto.id === 2);

console.log(encontrado);
// { id: 2, nome: "Teclado" }
```

JavaScript

Arrow function retornando um objeto

Uma arrow function também pode devolver um objeto. Para retornar um objeto diretamente, envolvemos suas chaves com parênteses.

Assim, o JavaScript interpreta { ... } como o objeto que deve ser retornado.

```
const cartoes = produtos.map(produto => ({
  id: produto.id,
  titulo: produto.nome
}));

console.log(cartoes);
// [
//   { id: 1, titulo: "Mouse" },
//   { id: 2, titulo: "Teclado" },
//   { id: 3, titulo: "Monitor" }
// ]
```

Firjan SENAI